

Building a CNN for Rock-Paper-Scissors Classification

Lorenzo Pacacussi

`lorenzopacacussi@unimi.it`

State university of Milano, Milan, Italy

Abstract. This project aims to build and compare three different convolutional neural networks (CNNs) for classifying images of hand gestures representing rock, paper, scissors. The three CNNs represent three increasing levels of complexity and were trained on the same standardized dataset containing 2188 images of hand gestures. The training was performed using a train test split of 70/30. Testing was performed using both the test set from the standardized dataset and two custom datasets containing 70 and 40 images respectively, which were collected by the author and contain more challenging images (e.g. with different backgrounds, lighting conditions, hand orientations, etc.). Results show that the most complete CNN achieved the best performance on the standardized test set, with higher validation accuracy than training accuracy, which suggests that the model is not overfitting and is able to generalize well to unseen data. However, the performance on the custom datasets was significantly lower, especially on the second custom dataset which contains images with different backgrounds where the model's performance is similar to a random classifier. Analyzing the results, it can be observed that the model is able to correctly classify images with simple backgrounds and good lighting conditions, but struggles with images that have more complex backgrounds or poor lighting conditions. Another critical aspect is the position of the fingers, error analysis revealed that most of the misclassifications contain finger positions that are similar to other classes.

1 Dataset Exploration

The main dataset used for this project is the Kaggle Rock-Paper-Scissors dataset [6], which contains 2,188 images of hand gestures representing rock, paper, and scissors. The dataset is organized into three folders, one for each class, and each folder contains images with similar orientations and lighting conditions. The images are provided in JPEG format with a resolution of 300×200 pixels.

In addition to the main dataset, two secondary custom datasets were collected by the author, containing 40 and 70 images respectively. These datasets were used to evaluate the performance of the CNNs on more challenging and diverse images. The first custom dataset includes hand gesture images captured under different lighting conditions and with varying rotations, while still maintaining

a homogeneous background. The second custom dataset contains images with more heterogeneous backgrounds, different camera angles, and greater variability overall. Both custom datasets are balanced, with approximately the same number of images for each class.

All datasets contain the same three classes: rock, paper, and scissors. However, the primary training dataset is highly homogeneous, as most images share similar orientations, poses, lighting conditions, and backgrounds. This homogeneity is reflected in the high performance achieved by the model on the training data. The same limitation also affects the validation and test sets, since they are generated by splitting the same original dataset used for training. As a result, these sets may not fully represent the variability encountered in real-world scenarios, making generalization to more diverse images more challenging.



Fig. 1. Sample images from the training dataset.

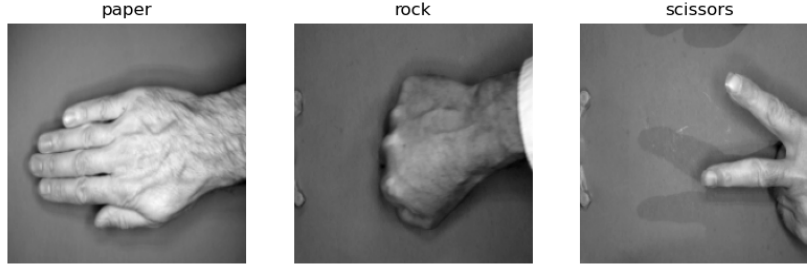
2 Methodology

2.1 Preprocessing

To speed up both processing and testing the images in the dataset were resized to 200x200 pixel and changed to greyscale, this allows to process only one third of the data for each image, since the color channels are reduced to one. This also means that the model will not learn any color-based feature, in this particular usecase this is a good thing since the classification is strictly shape based, color could not influence the decisions of the model and could even be a source of overfitting, since the training dataset is very homogeneous and the images all share the same green background.

Images were then normalized by dividing their pixel values by 255 to scale them to the range $[0, 1]$. This normalization step helps to improve the convergence of the CNN during training by ensuring that the input data is on a consistent scale. This step was implemented in the code as a layer within the model, but conceptually it is a preprocessing step that is applied to the input data before it is fed into the CNN.

The main dataset was split into a training set (70%), a validation set (15%), and a test set (15%). The training set was used to train the CNNs, while the validation set was used to tune hyperparameters and prevent overfitting. The



test set was reserved for evaluating the final performance of the models after training.

Data augmentation techniques were applied in the two most complex CNNs to prevent overfitting the augmentation techniques used were:

- Random rotation: Images were randomly rotated within a range of -36 to 36 degrees since the training data contained images with the same orientation, this was aimed at making the model able to recognize the actual hand gesture and not just the orientation of the fingers.
- Random zoom: Images were randomly zoomed in or out by up to 10% to help the model learn to recognize hand gestures at different distances from the camera and indirectly of different hand sizes.
- Random horizontal flip: Images were randomly flipped horizontally, this was important since the whole dataset contained images with the hand coming into frame from the right side.
- Random brightness adjustment: The brightness of the images was randomly adjusted by up to 40% to make the model more robust to varying lighting conditions.
- Random contrast adjustment: The contrast of the images was randomly adjusted by up to 40% to help the model learn to recognize hand gestures under different lighting conditions, this is not necessary for this dataset but this implementation needs to be robust enough to be usable in a less controlled environment.
- Random translation: Images were randomly translated horizontally and vertically by up to 10% to help the model learn to recognize hand gestures that are not perfectly centered in the frame.

These data augmentation steps have been implemented as layers within the CNN, this means that they are applied in real time during training allowing training to be more diverse, since for each epoch the same image will be randomly augmented in a different manner. This helps to prevent overfitting by exposing the model to a wider variety of training examples.

2.2 Model Architectures

Three different CNN architectures were implemented with growing complexities. The first model is a simple CNN with one convolutional layer, followed by a max-

pooling layer, a flattening layer and two fully connected layers one with a ReLU activation function and one with a softmax activation function for classification. The second model is a more complex CNN with the first layers being the data augmentation layers, two convolutional layers, each followed by a maxpooling layer, a flattening layer and three fully connected layers, two with a ReLU activation function and one with a softmax activation function for classification. The third model is the most complex one the first layer is the data augmentation layer, it has four convolutional layers, each followed by a maxpooling layer, a flattening layer and three fully connected layers, two with ReLU activation function and one with a softmax activation function for classification. After each of the dense layers a dropout layer with a rate of 0.2 was added to force generalization and prevent overfitting.

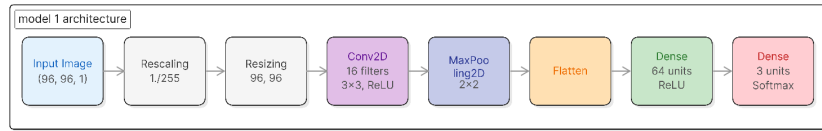


Fig. 2. Architecture of the first CNN model.

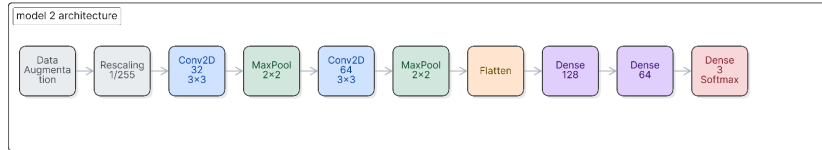


Fig. 3. Architecture of the second CNN model.

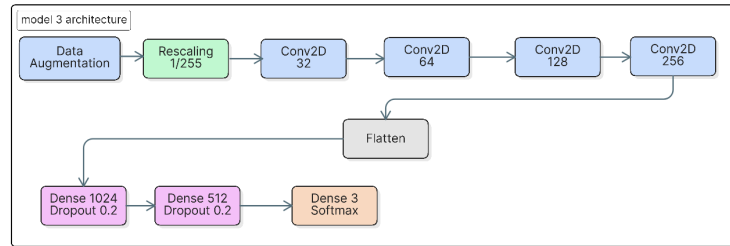


Fig. 4. Architecture of the third CNN model. Maxpooling layers are not shown for simplicity.

The convolutional layers use a 3×3 kernel size in all three models. What changes between the models is the number of filters: 16 filters in the first model, 32 and 64 filters in the second model, and 32, 64, 128, and 256 filters in the third model.

The purpose of convolutional layers is to learn local features from the input images. Increasing the number of filters increases the complexity and representational capacity of the model, allowing it to learn more complex features. However, a higher number of filters also increases the risk of overfitting. For this reason, data augmentation and dropout layers are introduced in the second and third models to improve generalization performance.

The max-pooling layers use a 2×2 pool size with a stride of 2. These layers downsample the feature maps produced by the convolutional layers, progressively reducing their spatial dimensions. This process helps the network learn increasingly higher-level features as depth increases. For example, the first convolutional layers typically learn simple patterns such as edges and corners, while deeper convolutional layers learn more complex representations such as shapes, textures, and object-specific patterns.

The fully connected layers perform the final classification based on the features extracted by the convolutional layers. The first fully connected layers use the ReLU activation function, which enables the network to learn non-linear relationships between features. The final fully connected layer uses a softmax activation function, which outputs a probability distribution over the three classes (rock, paper, and scissors) for each input image.

The loss function used for training the models is sparse categorical cross entropy since the target labels are provided as integers (0 for rock, 1 for paper, and 2 for scissors) and the output of the model is a probability distribution over the three classes. The simplified formula for sparse categorical cross entropy that is used in keras is:

$$L(y, \hat{y}) = -\log(\hat{y}_y)$$

Where: - y is the true class label for the input image, represented as an integer (0 for rock, 1 for paper, and 2 for scissors). - $\hat{y}_y$ is the predicted probability for the true class y of the i -th sample. This is obtained from the output of the softmax layer, which gives a probability distribution over the classes.

Since the training is done using batches of 32 images and the loss is averaged over the batch, the final loss for a batch of N samples can be expressed as:

$$L = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{y_i})$$

Where N is the batch size (32 in this case) and y_i is the true class label for the i -th sample in the batch.

The optimizer used for training the models is Adam, which is an adaptive learning rate optimization algorithm that combines the advantages of both momentum and RMSProp.

The first moment represents the exponentially weighted moving average of the gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

Where β_1 is the decay rate for the moving average of the gradients, typically set to 0.9. This means that the current gradient contributes 10% to the first moment, while the previous first moment contributes 90%.

The second moment tracks the exponentially weighted average of the squared gradients:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

Where β_2 is the decay rate for the moving average of the squared gradients, typically set to 0.999.

Since both m_t and v_t are initialized to zero, they are biased toward zero in the early training steps. To efficiently correct for this bias during implementation, the learning rate α is dynamically scaled at each time step t :

$$\alpha_t = \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}$$

Using this time-step adjusted learning rate along with the raw first and second moments, the final weight update rule is given by:

$$w_{t+1} = w_t - \alpha_t \frac{m_t}{\sqrt{v_t} + \epsilon}$$

where ϵ is a small constant added to avoid division by zero.

While adaptive learning rate is useful, the initial learning rate α is still an important hyperparameter that needs to be tuned for optimal performance. In this project, to find the best learning rate for the most complex model, a grid search was performed over common learning rates (0.003, 0.001, 0.0003, 0.0001, 0.00003, 0.00002, 0.00001) and the best performance was achieved with a learning rate of 0.001, which is a common default value for Adam and often works well in practice.

Early stopping was implemented to prevent overfitting by monitoring the validation loss during training. If the validation loss does not improve for a specified number of epochs (patience), training is stopped early. The learning rate was also reduced in case of a plateau in the validation loss, meaning that if the validation loss does not improve for a certain number of epochs, the learning rate is reduced by a factor (e.g., 0.1) to allow the model to converge to a better minimum.

The metrics used to evaluate the performance of the models during training and testing are accuracy and loss, both on the training and validation sets. When evaluating the final performance on the test sets (standardized and custom), the metrics used are accuracy, precision, recall, and F1-score. Accuracy measures the overall correctness of the model's predictions, while precision, recall, and

F1-score provide more detailed insights into the model's performance for each class, especially in cases where the classes may be imbalanced or when the cost of false positives and false negatives differs, although this is not the case in our dataset.

The confusion matrix is also used to visualize the performance of the model by showing the number of true positives, true negatives, false positives, and false negatives for each class. This helps to identify specific areas where the model may be struggling, such as misclassifying certain classes more frequently than others. The AUC was also computed to compute the area under the receiver operating characteristic (ROC) curve. The AUC provides a single scalar value that summarizes the model's ability to discriminate between classes across all possible classification thresholds. Since this is a multi-class classification problem, the AUC was computed using a one vs rest approach, where the AUC is calculated for each class against all the other classes.

References

1. Author, F.: Article title. *Journal* **2**(5), 99–110 (2016)
2. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) *CONFERENCE 2016, LNCS*, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). <https://doi.org/10.1007/1234567890>
3. Author, F., Author, S., Author, T.: Book title. 2nd edn. Publisher, Location (1999)
4. Author, A.-B.: Contribution title. In: 9th International Proceedings on Proceedings, pp. 1–2. Publisher, Location (2010)
5. LNCS Homepage, <http://www.springer.com/lncs>, last accessed 2023/10/25
6. Author, Julien de la Bruère-Terreault: Rock Paper Scissors Dataset. Kaggle, <https://www.kaggle.com/datasets/drgfreeman/rockpaperscissors/data> (2023)